Name : Mohit Gabra

Roll No. 2401033

Batch : T12

Subject : ASD&D

EXPERIMENT – 6

AIM : To study and Implement Agile Software Project lifecycle using Jira with DevOps Pipeline Integration.

LO MAPPED : LO1.

LO1 : Understand and apply Agile software development methodologies and project tracking tools to plan, manage and track the software project lifecycle.

THEORY :

Agile Software Development

Agile is an approach to software development in which work is carried out in short repeated cycles instead of one long phase. Requirements are broken into small pieces that can be delivered independently, and working software is produced at the end of every cycle. Because the product is reviewed after each cycle, a change in requirement can be accepted at any stage rather than only at the start of the project.

Agile Software Project Lifecycle

The Agile lifecycle repeats the same phases for every iteration. Requirements are collected and recorded in a backlog, a subset is selected during sprint planning, the selected items are developed and tested, the increment is reviewed, and the work is released. Items not selected remain in the backlog for a later iteration, so the project advances through a series of small completed increments.

Scrum

Scrum is the Agile framework used in this experiment. Work is organised into fixed-length iterations called sprints. The product backlog holds all known requirements, and the items chosen for the current iteration form the sprint backlog. Progress is tracked on a board during the sprint, and the sprint is formally closed when the selected items are complete.

Scrum Board

A Scrum board shows the issues of the active sprint arranged in columns, one column for each state of the workflow. The board used in this experiment has the columns To Do, In Progress, In Review and Done. Moving a card from one column to the next records the transition against the issue, so the board always shows the current state of the sprint.

User Stories and Story Points

A user story describes a requirement from the point of view of the person who will use the software, and is small enough to be finished within a single sprint. A task is technical work that does not describe user-facing behaviour, such as configuring a build server. Story points are a relative estimate of the effort a story needs, and adding the points of the selected stories gives the planned size of the sprint.

Jira

Jira is a project tracking tool used to manage Agile projects. Each requirement is recorded as an issue with a unique issue key, a type, an assignee, an estimate and a status. Issues are shown on the board and moved through the workflow as work proceeds. When a sprint is completed, Jira produces reports from the recorded data.

DevOps

DevOps is a set of practices that brings development and operations together so that software can be built, tested and released frequently and reliably. It depends on automation, so that steps which would otherwise be carried out by hand are performed by tools in the same order every time.

Continuous Integration and Continuous Deployment

Continuous Integration is the practice of adding code changes to a shared repository regularly, where each change is automatically built and checked so that errors are found early. Continuous Deployment extends this by automatically delivering the code that passes those checks to the target environment.

Git and GitHub

Git is a distributed version control system that records the history of changes to the source code and allows each feature to be developed on its own branch. GitHub is a remote hosting service for

Git repositories, and it is the repository from which the automation server obtains the application files and the pipeline definition.

Jenkins

Jenkins is an open-source automation server that performs the stages of the pipeline. A Declarative Pipeline defines those stages in a file named Jenkinsfile which is stored in the repository with the application source code, so the pipeline is versioned along with the code. Jenkins can also be configured to start a build by itself when the repository changes, which removes the need to press Build Now.

Poll SCM

Poll SCM is a Jenkins trigger that checks the repository at a fixed interval and starts a build when a new commit is found. It achieves the same result as a GitHub webhook, and is used here because Jenkins is running on localhost and cannot receive an incoming webhook from GitHub.

Docker

Docker packages an application together with the environment it needs into an image, which can then be run as a container on any machine that has Docker installed. Because the image contains the web server as well as the application files, the application behaves the same way wherever the container is run.

Integration of Jira with the DevOps Pipeline

Jira and the pipeline are linked through the issue key. Each branch is named after the issue it implements and each commit message begins with the same issue key, so the code change that satisfies a requirement can be identified from the Jira issue and the build that carried it. The team also agreed a definition of done which requires the automated build to succeed before an issue may be moved to Done, so the state of the board reflects the state of the pipeline.

Agile and DevOps Lifecycle Flow:

Requirement recorded in Jira → Story estimated and pulled into the sprint → Branch created and code committed with the issue key → Pull request reviewed and merged → Jenkins detects the change and runs the pipeline → Application containerised and deployed → Issue verified and moved to Done → Sprint completed and reports generated

EXECUTION :

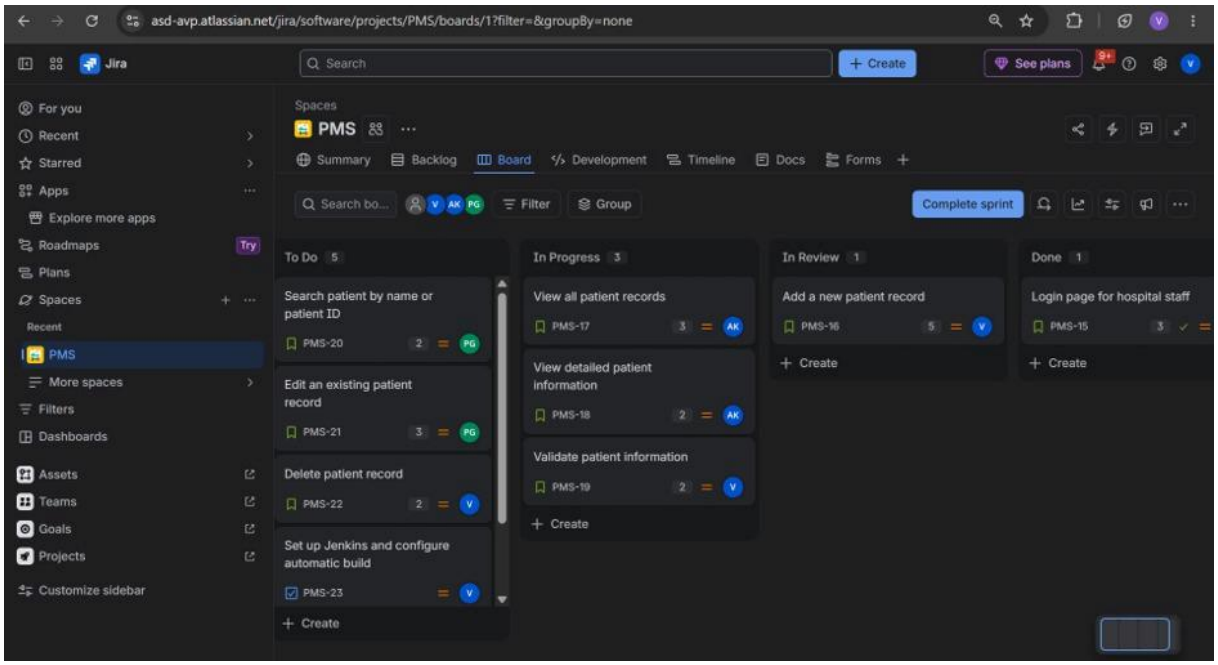
The application used throughout this experiment is the Patient Management System, a static web application consisting of index.html, app.js and style.css. It was planned in Jira, version controlled in the GitHub repository MohitGabra25 /patient-management-system, built automatically by Jenkins and deployed in a Docker container.

Step 1: Create a Jira Account and a Scrum Project

A free Atlassian account was created and a new software project was added, choosing the Scrum project template so that the work could be organised into sprints. The project was named Patient Management System and was given the key PMS, which is the prefix of every issue key in the project.

Step 2: Understand the Scrum Board

The Scrum board of the project was examined. The board shows the issues of the active sprint in columns, one for each workflow state — To Do, In Progress, In Review and Done. Cards are dragged from one column to the next as work progresses, and the count above each column shows how many issues are currently in that state. The Complete sprint button at the top right is used to close the sprint once the selected issues are finished.



Step 3: Create User Stories

Each requirement of the application was raised using Create Issue. Requirements describing behaviour visible to the hospital staff using the application were raised as Stories, and technical work such as configuring the build server and containerising the application was raised as Tasks. Every issue was assigned to a team member and the stories were estimated in story points.

Table 1: Sprint Backlog of Project PMS

Issue Key	Type	Summary	Points	Owner
PMS-15	Story	Login page for hospital staff	3	Aryan Kashid
PMS-16	Story	Add a new patient record	5	Vineet Gajwani
PMS-17	Story	View all patient records	3	Aryan Kashid
PMS-18	Story	View detailed patient information	2	Aryan Kashid
PMS-19	Story	Validate patient information	2	Vineet Gajwani
PMS-20	Story	Search patient by name or patient ID	2	Mohit Gabra
PMS-21	Story	Edit an existing patient record	3	Mohit Gabra
PMS-22	Story	Delete patient record	2	Vineet Gajwani
PMS-23	Task	Set up Jenkins and configure automatic build	—	Vineet Gajwani
PMS-24	Task	Containerise application using Docker	—	Mohit Gabra
Total Story Points			22	

The twenty two points were divided between the three members as eight for Aryan, nine for Vineet and five for Mohit, which kept the load of the sprint roughly even.

Step 4: Create and Start the Sprint

A sprint named PMS Sprint 1 was created and the issues of the backlog were dragged into it. Starting the sprint moved the selected issues onto the board and began the iteration, after which the board became the working view for the team.

Step 5: Create a GitHub Account and Repository

A GitHub repository named patient-management-system was created under the account MohitGabra25 to hold the application source code. The repository was initially empty and the project files were pushed into it in Step 8.

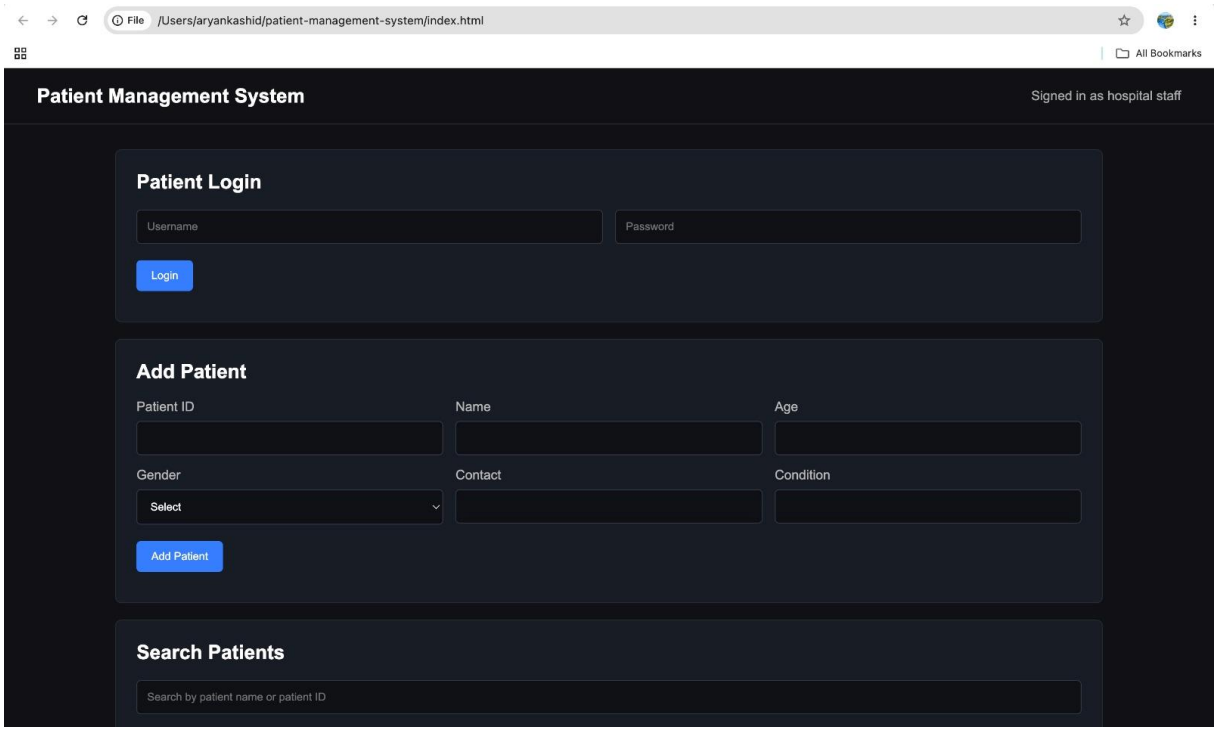
Step 6: Install Git

Git was installed on the development machine and the installation was verified from the terminal.

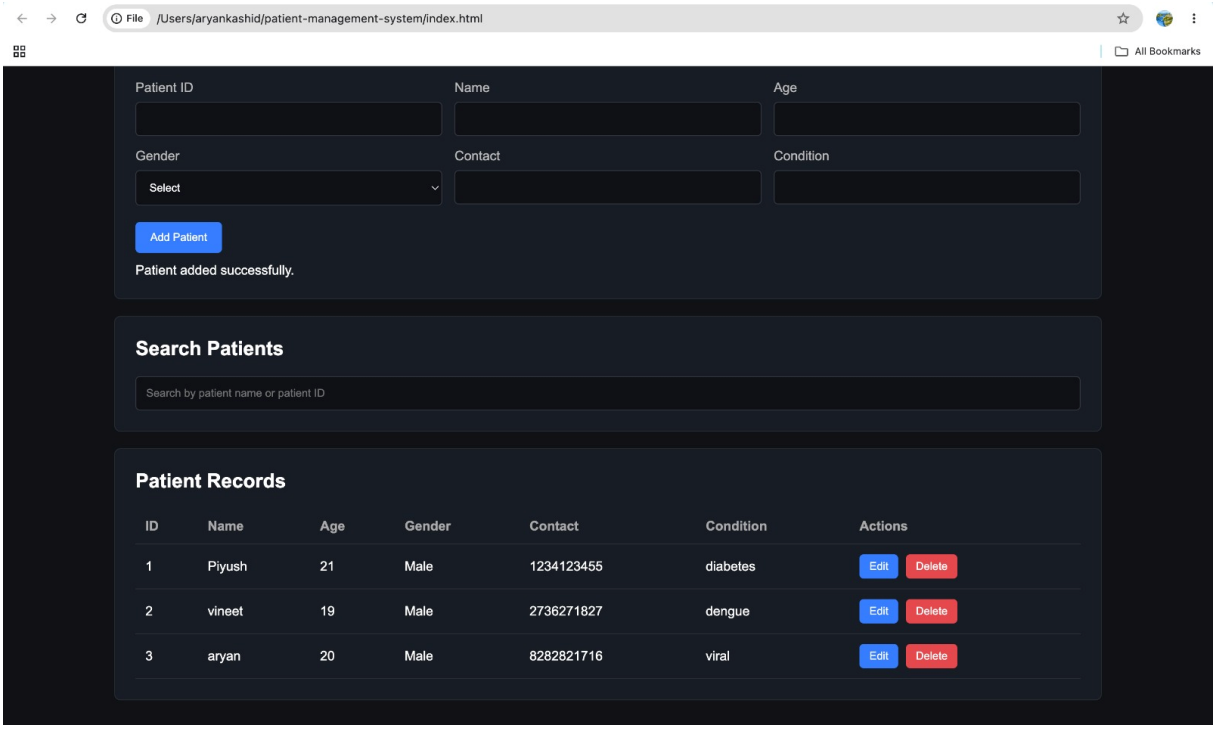
```
git --version
```

Step 7: Create the Application

The Patient Management System was written as a static web application using HTML, CSS and JavaScript. The interface provides a hospital-staff login, a form to add a patient with fields for Patient ID, Name, Age, Gender, Contact and Condition, a search box, and a table of patients with Edit and Delete actions, which together implement the user stories of Table 1.



After a patient is added the form confirms the addition and the new record appears in the Patient Records table below, alongside the records already stored for the hospital.

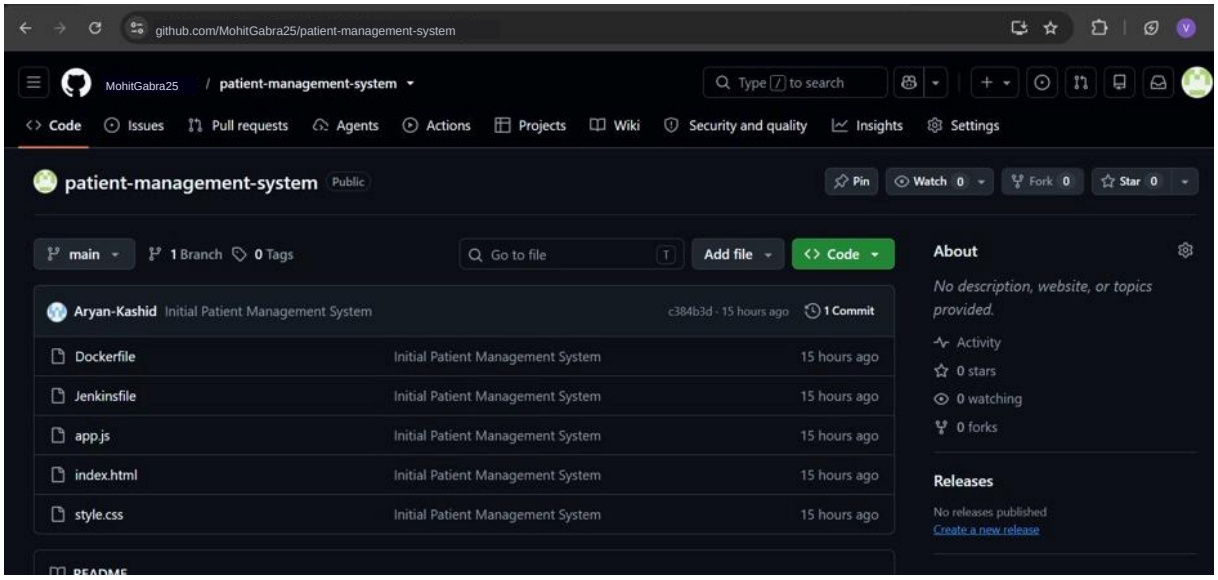


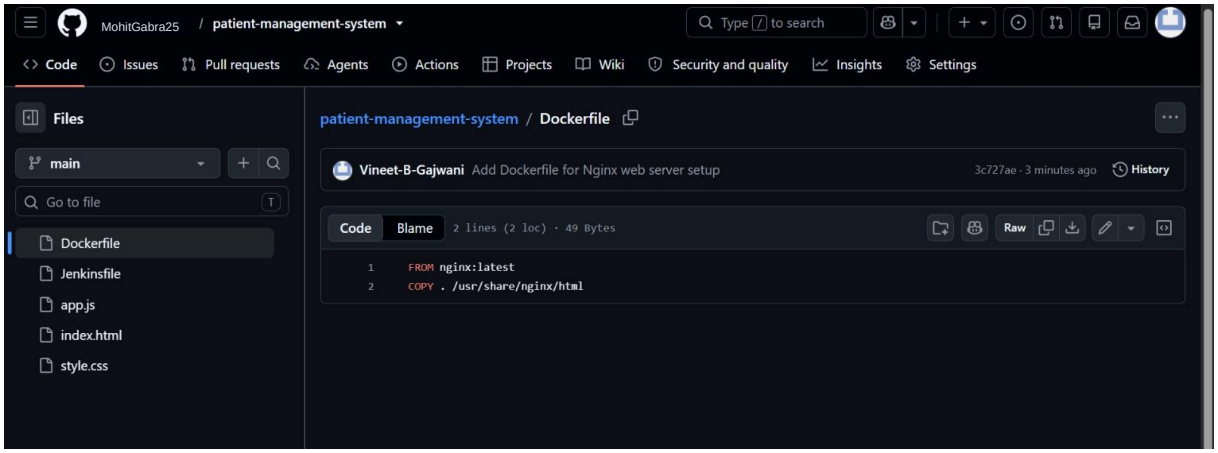
Step 8: Upload the Code to GitHub

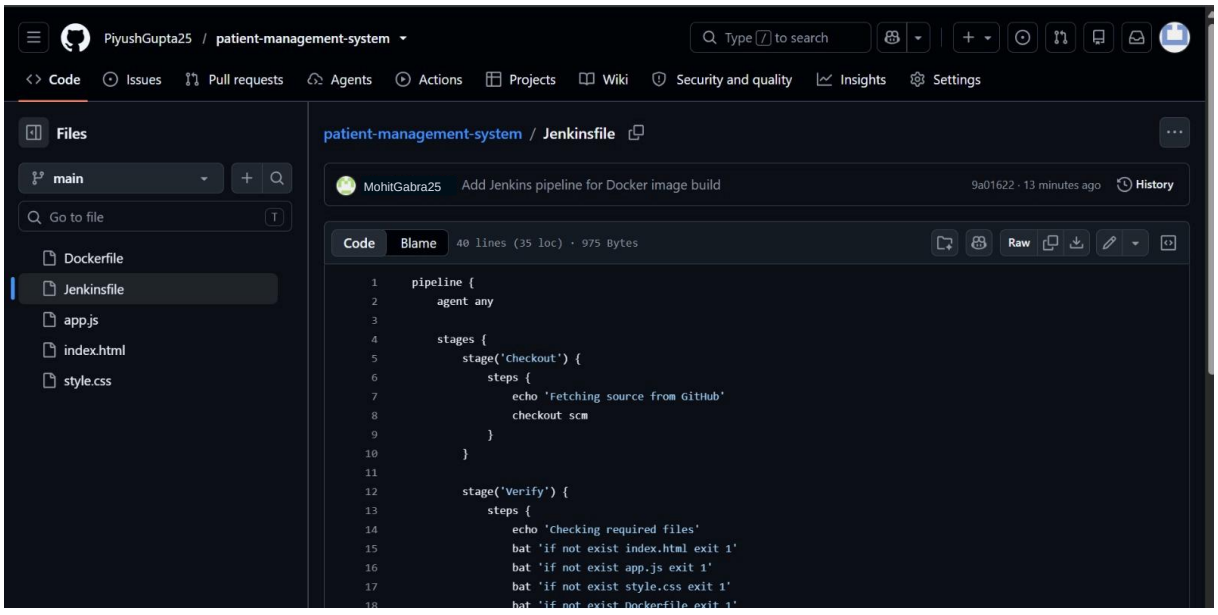
The project folder was initialised as a Git repository, the files were staged and committed, the GitHub repository was added as the remote named origin, and the code was pushed. Each commit message begins with the key of the Jira issue it implements, which is what links the repository history back to the board.

```
git init
git add .
git commit -m "PMS-15 Login page completed"
git remote add origin https://github.com/MohitGabra25    /patient-management-
system.git
git push origin main
```

Every story was developed on its own branch named after its issue key and merged into main through a pull request reviewed by another team member. A Dockerfile was later added to the repository so that the application could be containerised, and a Jenkinsfile was added alongside it to define the automated pipeline.


github.com/MohitGabra25/patient-management-system
MohitGabra25 / patient-management-system
Type / to search
Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings
patient-management-system Public
Pin Watch 0 Fork 0 Star 0
main 1 Branch 0 Tags
Go to file
Add file
Code
Aryan-Kashid Initial Patient Management System
c384b3d · 15 hours ago
1 Commit
Dockerfile Initial Patient Management System 15 hours ago
Jenkinsfile Initial Patient Management System 15 hours ago
app.js Initial Patient Management System 15 hours ago
index.html Initial Patient Management System 15 hours ago
style.css Initial Patient Management System 15 hours ago
About
No description, website, or topics provided.
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release


MohitGabra25 / patient-management-system
Type / to search
Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings
Files
main
Go to file
Dockerfile
Jenkinsfile
app.js
index.html
style.css
patient-management-system / Dockerfile
Vineet-B-Gajwani Add Dockerfile for Nginx web server setup
3c727ae · 3 minutes ago
History
Code Blame 2 lines (2 loc) · 49 Bytes
Raw
1 FROM nginx:latest
2 COPY . /usr/share/nginx/html


PiyushGupta25 / patient-management-system
Type / to search
Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings
Files
main
Go to file
Dockerfile
Jenkinsfile
app.js
index.html
style.css
patient-management-system / Jenkinsfile
MohitGabra25 Add Jenkins pipeline for Docker image build
9a01622 · 13 minutes ago
History
Code Blame 40 lines (35 loc) · 975 Bytes
Raw
pipeline {
agent any
stages {
stage('Checkout') {
steps {
echo 'Fetching source from GitHub'
checkout scm
}
}
stage('Verify') {
steps {
echo 'Checking required files'
bat 'if not exist index.html exit 1'
bat 'if not exist app.js exit 1'
bat 'if not exist style.css exit 1'

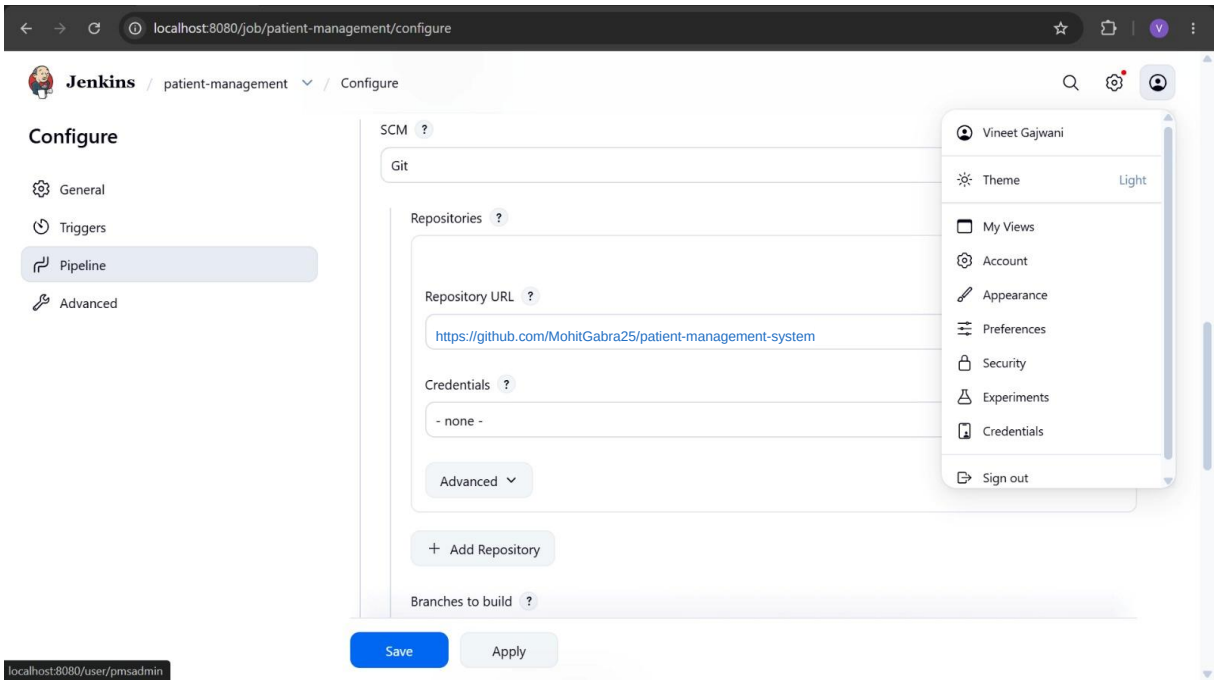
Step 9: Install Jenkins

Jenkins was installed and started on the development machine, and the setup wizard was completed by unlocking Jenkins with the generated administrator password, installing the suggested plugins and creating an admin user. The Jenkins dashboard is served at port 8080 of the local machine.

```
http://localhost:8080
```

Step 10: Connect Jenkins with GitHub

A Jenkins pipeline job named patient-management was created and its Source Code Management section was set to Git, with the repository URL pointing to https://github.com/MohitGabra25 /patient-management-system so that Jenkins fetches the application from GitHub rather than from a local folder.



Step 11: Build the Project

The Jenkinsfile committed with the source code defines three stages: Checkout, which fetches the source from GitHub; Verify, which checks that index.html, app.js, style.css, the Dockerfile and the Jenkinsfile are all present; and Build Docker image, which builds the container image once the required files are confirmed. Because Jenkins is running on Windows, the pipeline uses bat steps rather than sh.

```
pipeline {
    agent any
```

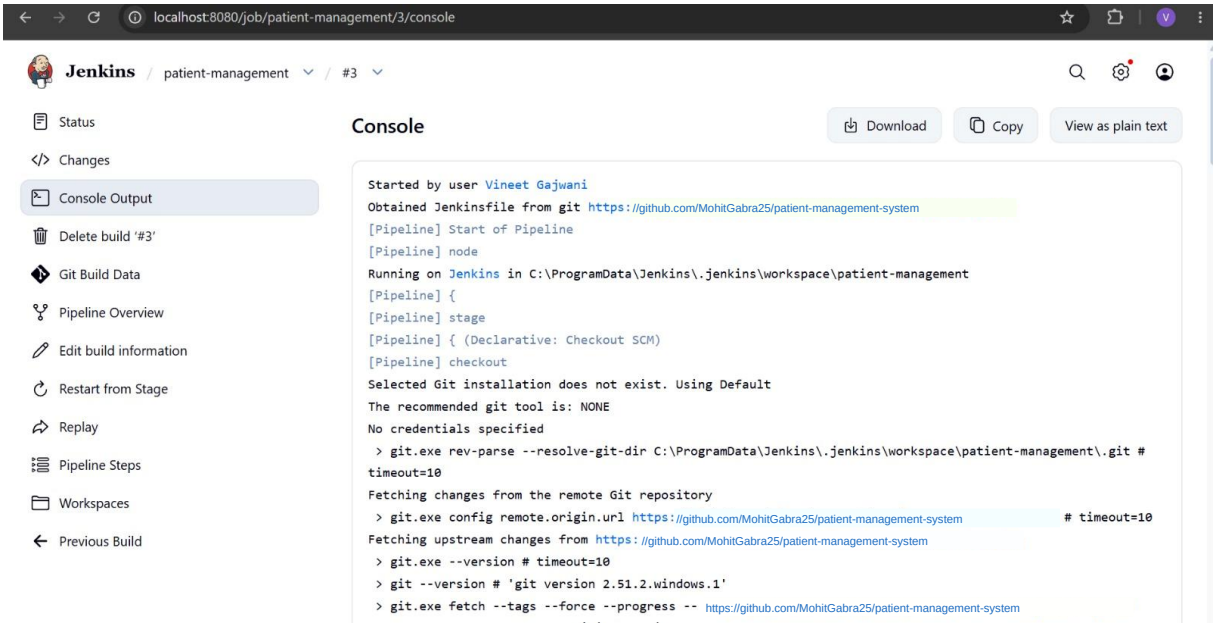
```
    stages {
        stage('Checkout') {
            steps {
                echo 'Fetching source from GitHub'
                checkout scm
            }
        }

        stage('Verify') {
            steps {
                echo 'Checking required files'
                bat 'if not exist index.html exit 1'
                bat 'if not exist app.js exit 1'
                bat 'if not exist style.css exit 1'
                bat 'if not exist Dockerfile exit 1'
                bat 'if not exist Jenkinsfile exit 1'
            }
        }

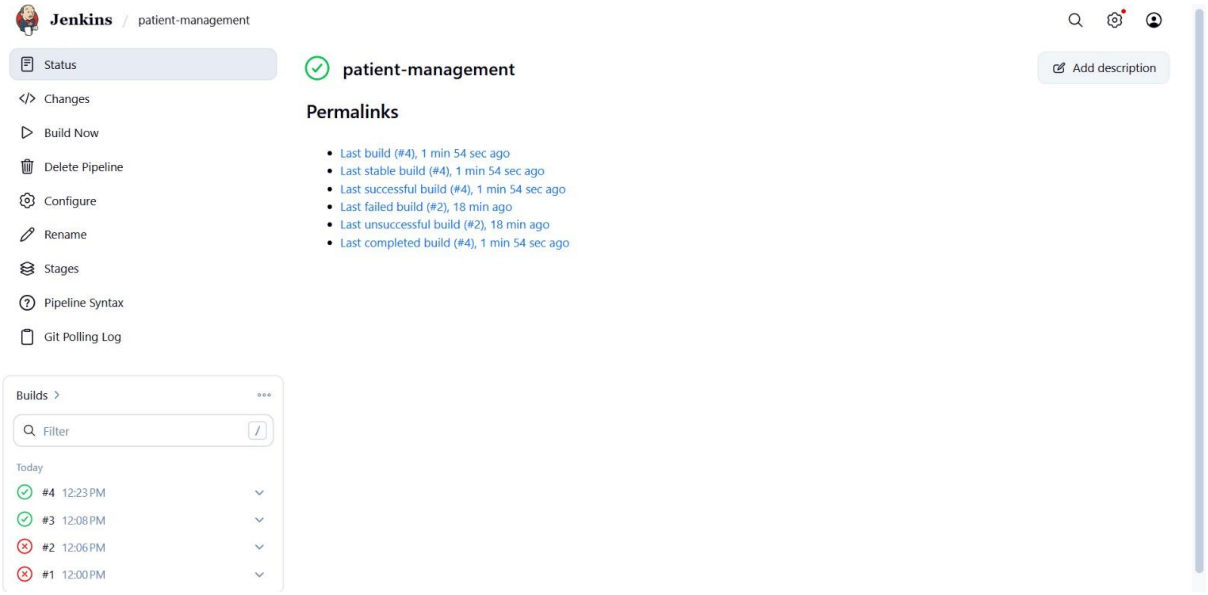
        stage('Build Docker Image') {
            steps {
                echo 'Building Docker image'
                bat 'docker build -t patient-management .'
            }
        }
    }

    post {
        success { echo 'SUCCESS - Build completed' }
        failure { echo 'FAILURE - Check console output' }
    }
}
```

The first two builds, started before the Dockerfile existed in the repository, failed at the Verify stage. Once the Dockerfile was pushed, the job was run again with Build Now: the console output below, started manually by team member Vineet Gajwani, shows the repository being fetched and the build completing as #3.

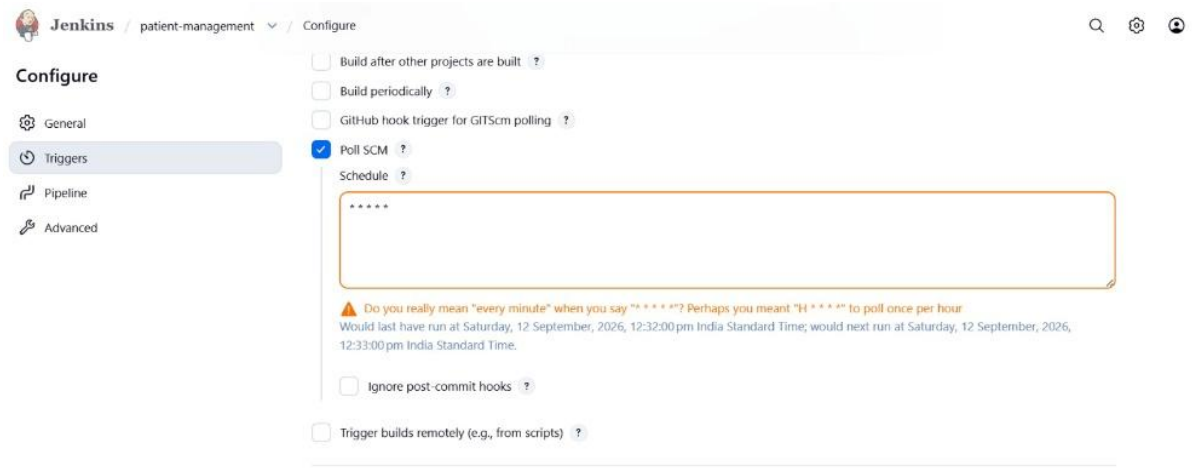


The Status page confirms build #3 as the last successful and last stable build at that point, with the two earlier failed attempts also listed in the build history.

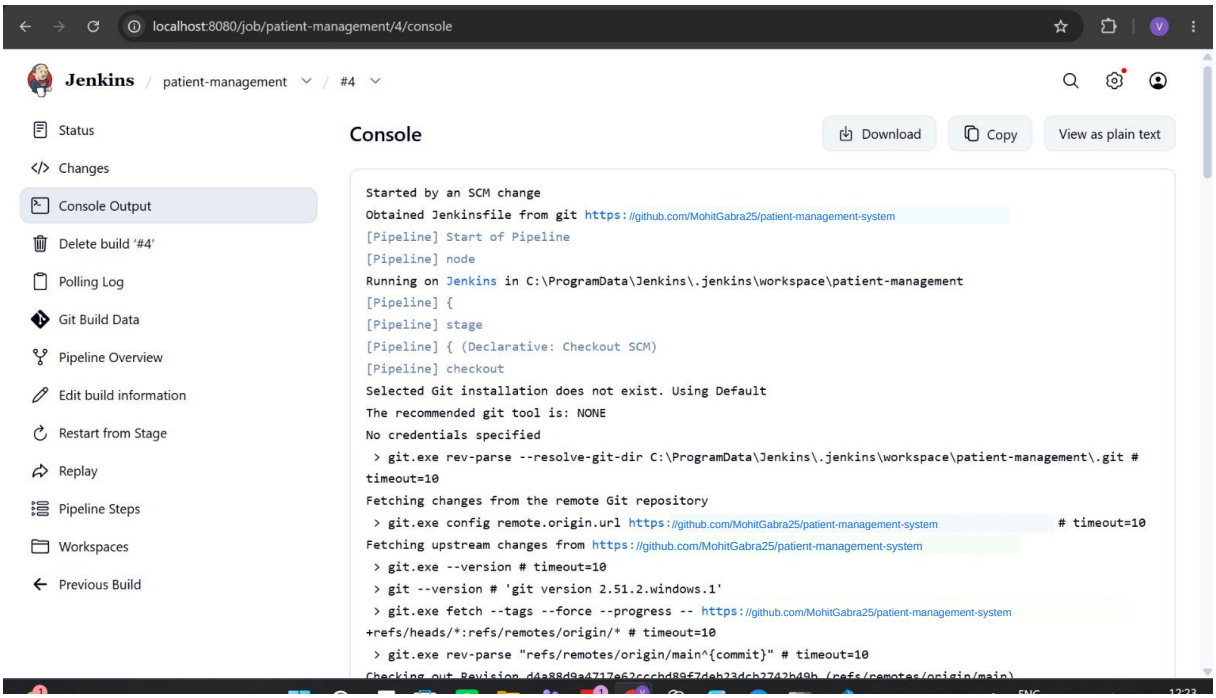


Step 12: Add Automatic Build

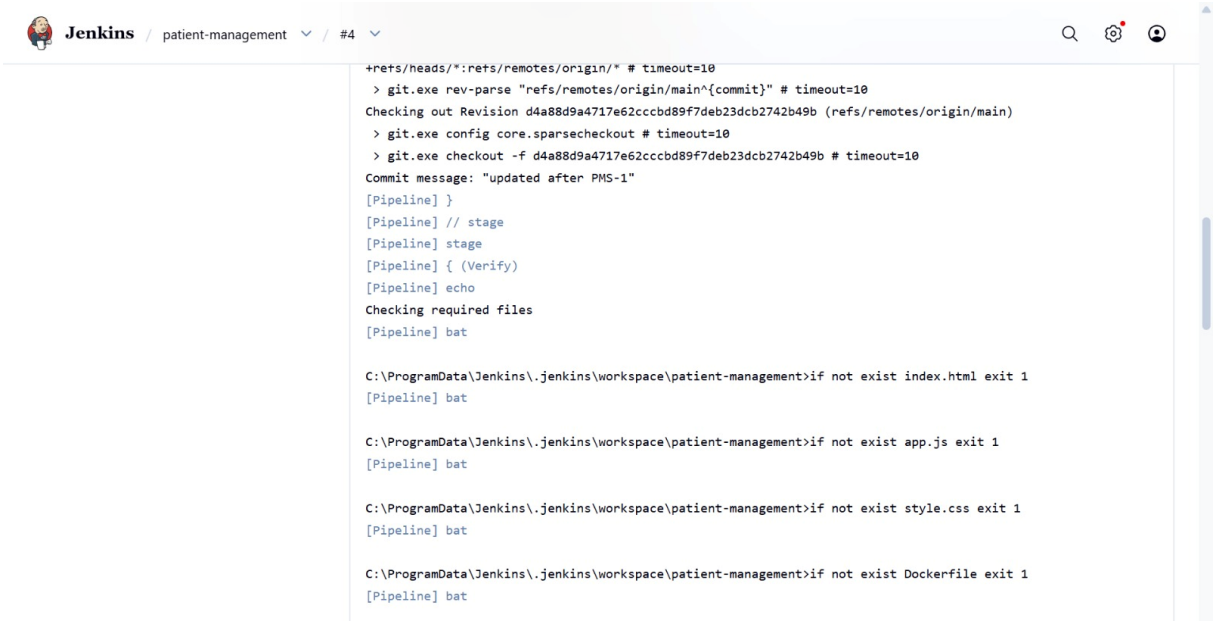
So that a build no longer has to be started by hand, an automatic trigger was configured. Because Jenkins is running on localhost it cannot receive an incoming webhook from GitHub, so Poll SCM was used instead, with a schedule of * * * * * so that Jenkins checks the repository every minute and starts a build whenever a new commit is found.



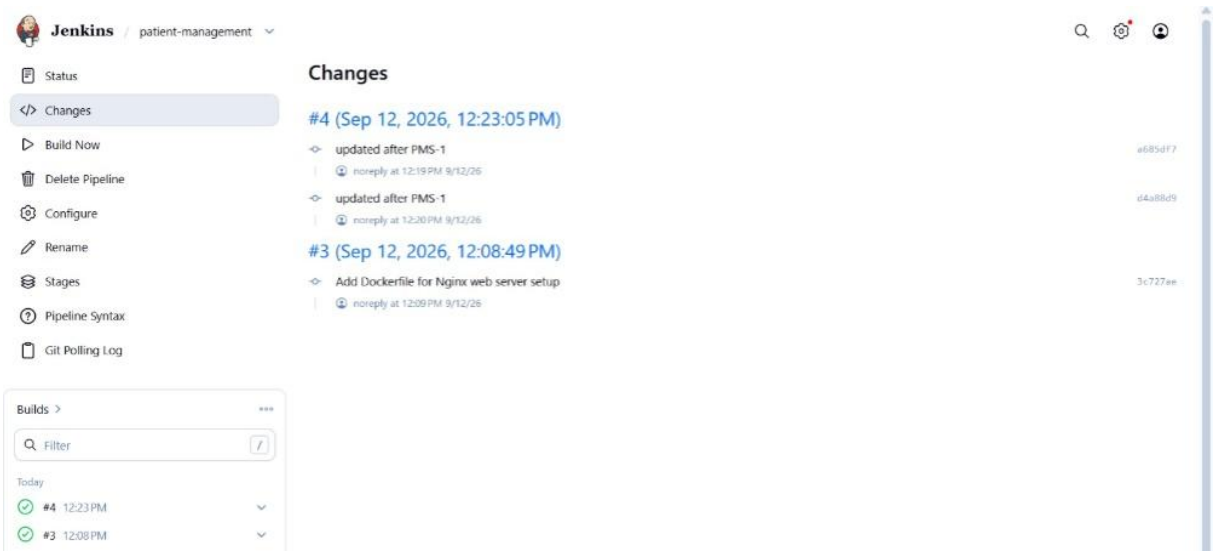
A commit referencing issue PMS-1 was then pushed to the repository without opening Jenkins. The next build, #4, started on its own: its console output records that it was started by an SCM change, together with the revision and the commit message that caused it.



Further down the same console output, the Verify stage confirms that index.html, app.js, style.css and the Dockerfile were all found before the build was allowed to proceed.

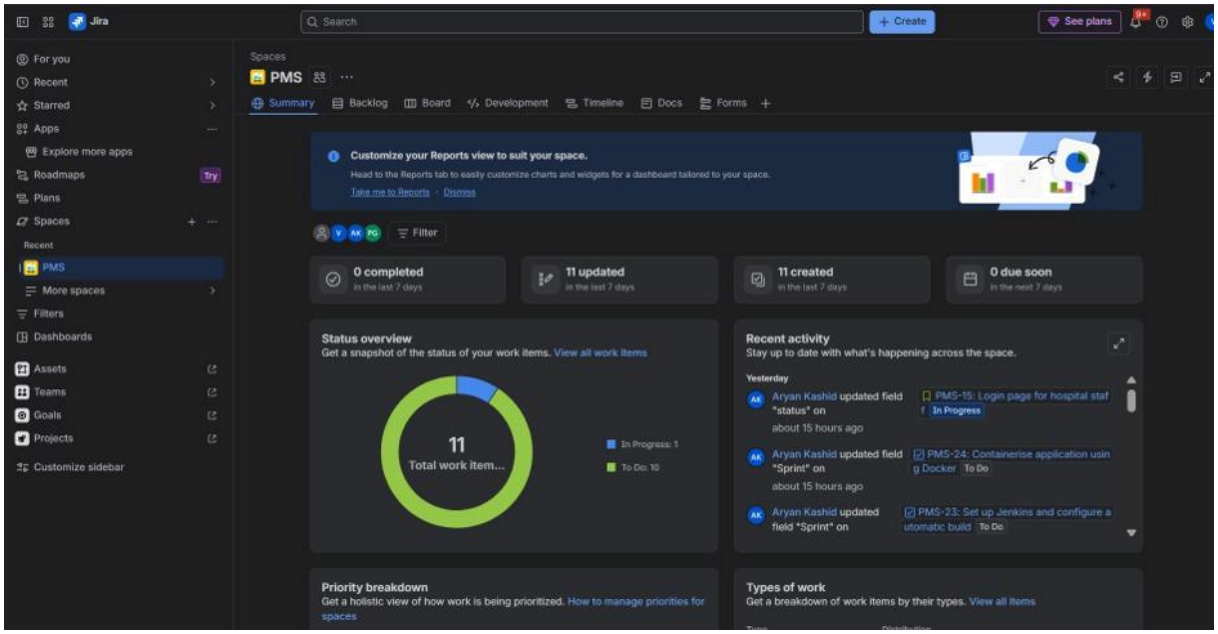


The Changes page for the job lists the commits responsible for the two most recent builds — the Dockerfile addition that produced build #3, and the two PMS-1 commits that triggered build #4 automatically.



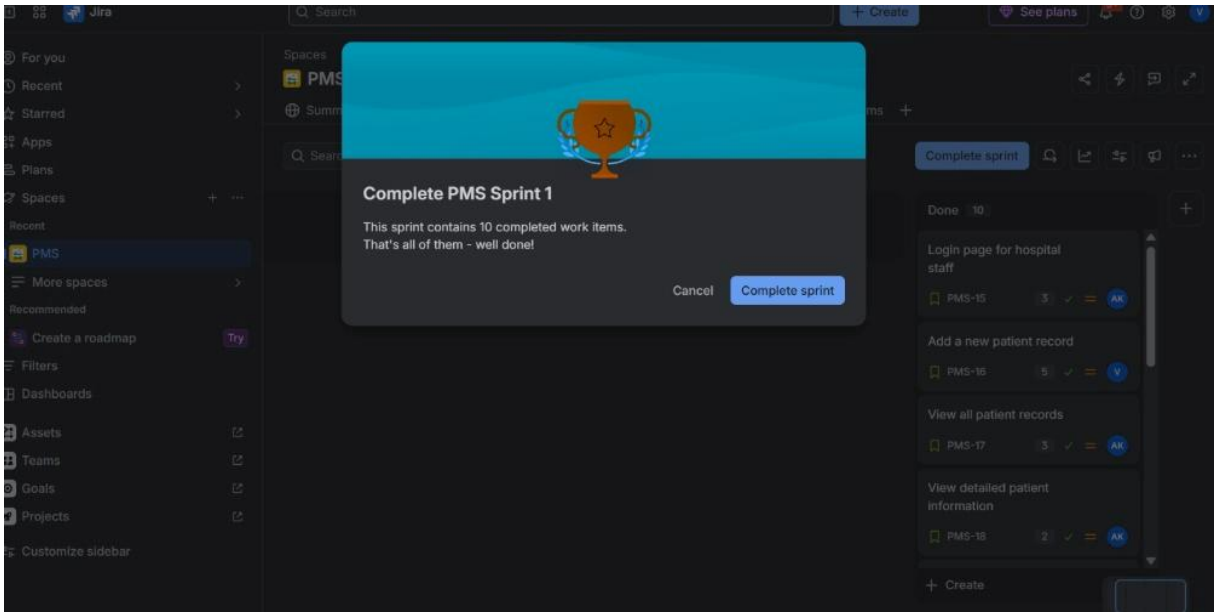
Step 13: Move the Stories in Jira

As each story was completed it was moved through the workflow on the board, from To Do to In Progress when development started, to In Review when the pull request was raised for a teammate to review, and to Done after the review, the merge and a successful Jenkins build.

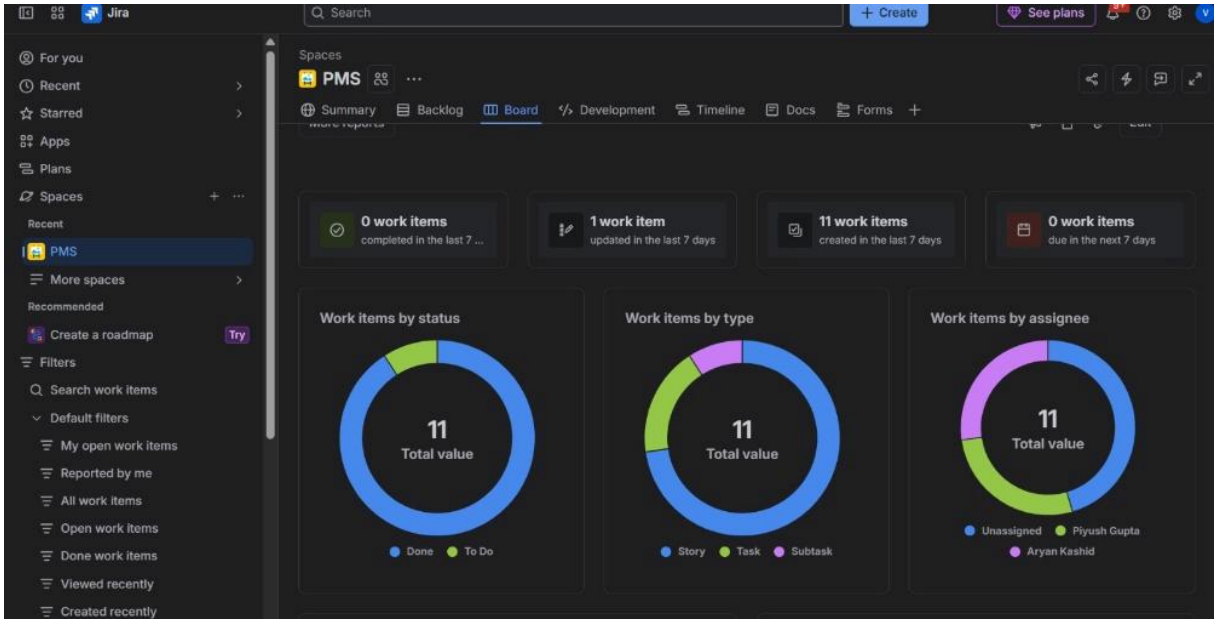


Step 14: Complete the Sprint

When the issues of the sprint had reached Done, the sprint was closed with Complete sprint. Jira confirmed that the sprint contained ten completed work items, with the Done column showing issues such as PMS-15, PMS-16, PMS-17 and PMS-18 alongside their story points and assignees.



Closing the sprint allows Jira to produce reports from the data recorded during the iteration. The project summary shows eleven work items in total, grouped by status, by type and by assignee, which gives the distribution of the completed work across the team.



Step 15: Docker Deployment

Docker was installed and a Dockerfile was added to the repository. The image is based on the official nginx image and copies the application files into the directory from which nginx serves content, so the resulting container serves the application without any further configuration.

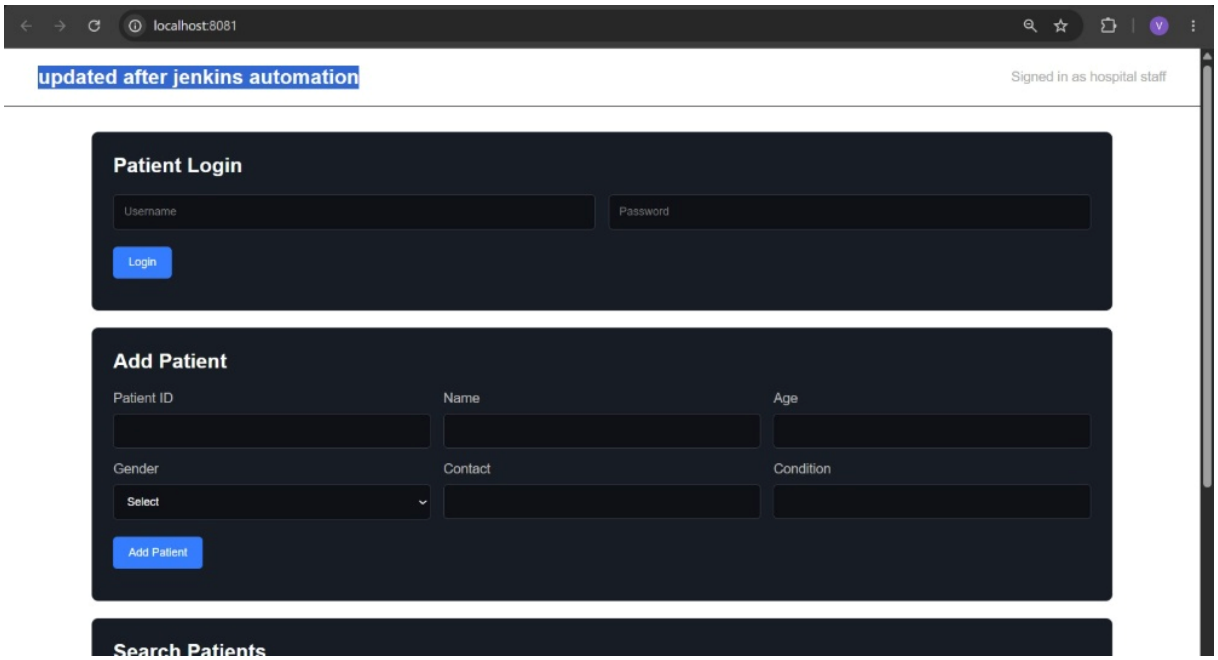
```
FROM nginx:latest
COPY . /usr/share/nginx/html
```

The image was then built and run as a container. Port 8081 of the host is mapped to port 80 of the container, because port 8080 on this machine is already occupied by Jenkins.

```
docker build -t patient-management .
docker run -p 8081:80 patient-management
```

```
C:\Users\Vineet Gajwani>docker run -p 8081:80 patient-management
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/09/12 06:56:22 [notice] 1#1: using the "epoll" event method
2026/09/12 06:56:22 [notice] 1#1: nginx/1.29.2
2026/09/12 06:56:22 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/09/12 06:56:22 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/09/12 06:56:22 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/09/12 06:56:22 [notice] 1#1: start worker processes
2026/09/12 06:56:22 [notice] 1#1: start worker process 29
2026/09/12 06:56:22 [notice] 1#1: start worker process 30
2026/09/12 06:56:22 [notice] 1#1: start worker process 31
2026/09/12 06:56:22 [notice] 1#1: start worker process 32
2026/09/12 06:56:22 [notice] 1#1: start worker process 33
2026/09/12 06:56:22 [notice] 1#1: start worker process 34
2026/09/12 06:56:22 [notice] 1#1: start worker process 35
2026/09/12 06:56:22 [notice] 1#1: start worker process 36
2026/09/12 06:56:22 [notice] 1#1: start worker process 37
2026/09/12 06:56:22 [notice] 1#1: start worker process 38
2026/09/12 06:56:22 [notice] 1#1: start worker process 39
2026/09/12 06:56:22 [notice] 1#1: start worker process 40
2026/09/12 06:56:22 [notice] 1#1: start worker process 41
2026/09/12 06:56:22 [notice] 1#1: start worker process 42
2026/09/12 06:56:22 [notice] 1#1: start worker process 43
2026/09/12 06:56:22 [notice] 1#1: start worker process 44
2026/09/12 06:56:22 [notice] 1#1: start worker process 45
2026/09/12 06:56:22 [notice] 1#1: start worker process 46
```

Opening http://localhost:8081 in a browser shows the Patient Management System running inside the container, with a heading updated by the automated build confirming that the deployed version reflects the latest commit.



CONCLUSION :

The experiment successfully demonstrated the Agile software project lifecycle using Jira integrated with a DevOps pipeline. A Scrum project named Patient Management System (PMS) was created, and user stories and technical tasks were managed through PMS Sprint 1. The application was stored in the GitHub repository MohitGabra25 /patient-management-system, built automatically using Jenkins with Poll SCM, and containerized with Docker using nginx on port 8081. Overall, the experiment provided practical knowledge of Scrum planning, Jira project management, Git and GitHub workflow, Jenkins automation, Docker deployment, and Agile-DevOps integration.